Claude

Sunday, October 11, 2026 · CLAUDE

https://github.com/SETAPESU26/17_wordle
for games that do not have a UI, you can submit a screen recording of the terminal with the game running and clearly visible.
pls make sure all the tasks mentioned are clearly showcased in your videos (for eg: timer, score updates, etc)
pls ask everyone to submit a pdf of the chat history and not a link since i won't be able to view the link
So basically, i need to run this whole thing in terminal, cmd whatever, give me all the commands to run it starting from git clone, and also give the necessary changes to be done to the code, give the code changes prompt by prompt
Scenario 17 — Wordle
A terminal Wordle-style game with duplicate-letter scoring and guess history.
Provided files

Here's the whole flow, start to finish.

0. Setup + "before" video

Fork `SETAPESU26/17_wordle` to your own account on GitHub first (that's how you avoid raising a PR to the main repo), then:

bash

```
git clone https://github.com/<your-username>/17_wordle.git
cd 17_wordle
python main.py
```

(use `py main.py` if `python` isn't recognised on Windows)

What to record in the before video (10 sec):

bash

```
python -c "from feedback import evaluate;
print(evaluate('crane','eerie'))"
```

Prints `['yellow', 'yellow', 'yellow', 'gray', 'green']`. That's the bug: crane has only one `e` and the last `e` already took it as green, so the first two should be gray and `r` is the only yellow.

Then `python main.py`, type `abc` a few times (invalid input eats a turn, second bug), then `q`.

Task 1: duplicate-letter feedback

Prompt:

> Read feedback.py. evaluate() marks a letter yellow if it's anywhere in the target, even when copies are already used up. Rewrite it with standard Wordle rules: greens first, then each remaining target letter can satisfy only one yellow. Keep the same signature and return a list of "green"/"yellow"/"gray". Explain the change briefly.

`feedback.py`:

python

```
def evaluate(target, guess):
    result = ["gray"] * len(guess)
    left = {}
    for i, ch in enumerate(guess):
        if ch == target[i]:
            result[i] = "green"
        else:
            left[target[i]] = left.get(target[i], 0) + 1
    for i, ch in enumerate(guess):
        if result[i] == "green":
            continue
```

```
        if left.get(ch, 0) > 0:
            result[i] = "yellow"
            left[ch] -= 1
    return result
```

Re-run the one-liner. You should get `['gray', 'gray', 'yellow', 'gray', 'green']`.

bash

```
git add feedback.py
git commit -m "task 1: fix duplicate letter feedback"
```

Task 2: win/loss, 6-guess limit, invalid guesses are free

Prompt:

> In game.py, run() uses `for _ in range(6)` so invalid guesses use up a turn. Switch to a loop that only counts accepted guesses (max 6), show guesses left, handle win, loss (reveal the word) and quitting with q, and have run() return "won"/"lost"/"quit". Don't touch the other files.

`game.py`:

python

```
import random
from words import WORDS
from feedback import evaluate


class WordleGame:
    def __init__(self, length=5):
        self.length = length
        self.target = random.choice([w for w in WORDS if len(w) ==
length])
        self.history = []
        self.max_tries = 6
        self.result = None

    def run(self):
        print(f"Wordle - {self.length} letters, {self.max_tries}
guesses. q to quit.")
        while len(self.history) < self.max_tries:
            left = self.max_tries - len(self.history)
            guess = input(f"[{left} left] > ").strip().lower()
            if guess == "q":
                print("quit. the word was:", self.target)
                self.result = "quit"
                return self.result
            if len(guess) != self.length or not guess.isalpha():
```

```
                print(f"enter a {self.length} letter word, letters
only.")
                continue
            feedback = evaluate(self.target, guess)
            self.history.append((guess, feedback))
            print(" ".join(feedback))
            if guess == self.target:
                print(f"solved in {len(self.history)}!")
                self.result = "won"
                return self.result
        print("out of guesses. the word was:", self.target)
        self.result = "lost"
        return self.result
```

bash

```
git add game.py
git commit -m "task 2: win/loss handling and guess limit"
```

Task 3: 4/5/6 letter modes

Prompt:

> Add 4 and 6 letter modes. Add some 4 and 6 letter words to words.py (include a few with repeated letters). main.py should ask for the length (4/5/6), re-ask on bad input, and pass it to WordleGame. Make WordleGame raise ValueError for unsupported lengths. Confirm evaluate() doesn't assume 5.

`words.py`:

python

```
WORDS = [
    "apple", "array", "eerie", "crane", "grape",
    "stare", "steam", "stone", "plant", "flame",
    "moon", "book", "door", "tree", "cake", "jazz",
    "coffee", "summer", "planet", "garden", "bridge", "silver",
]
```

`game.py`, change only these bits: add `LENGTHS = (4, 5, 6)` under the imports, and at the top of `__init__`:

python

```
        if length not in LENGTHS:
            raise ValueError("length must be 4, 5 or 6")
```

`main.py`:

python

```
from game import WordleGame


def pick_length():
    while True:
        raw = input("word length (4/5/6): ").strip()
        if raw in ("4", "5", "6"):
            return int(raw)
        print("pick 4, 5 or 6")


if __name__ == "__main__":
    WordleGame(pick_length()).run()
```

bash

```
git add words.py game.py main.py
git commit -m "task 3: 4/5/6 letter modes"
```

Task 4: history + session summary

Prompt:

> After every accepted guess, print the full history so far, readable without colours (green = [E], yellow = (E), gray = plain). Invalid guesses must not add history or reprint it. Add a per-game summary, and in main.py a play-again loop that prints a session summary at the end: games played, wins/losses/quits, average guesses on wins, and one line per game. Keep it modular, no files or extra libraries.

`game.py` (full file now):

python

```
import random
from words import WORDS
from feedback import evaluate

LENGTHS = (4, 5, 6)


def render(guess, feedback):
    parts = []
    for ch, c in zip(guess.upper(), feedback):
        if c == "green":
            parts.append(f"[{ch}]")
        elif c == "yellow":
            parts.append(f"({ch})")
        else:
            parts.append(f" {ch} ")
```

```
    return " ".join(parts)


class WordleGame:
    def __init__(self, length=5):
        if length not in LENGTHS:
            raise ValueError("length must be 4, 5 or 6")
        self.length = length
        self.target = random.choice([w for w in WORDS if len(w) ==
length])
        self.history = []
        self.max_tries = 6
        self.result = None

    def show_history(self):
        print()
        for n, (guess, feedback) in enumerate(self.history, 1):
            print(f"{n}. {render(guess, feedback)}")
        print()

    def show_summary(self):
        print("---- game summary ----")
        print("word   :", self.target)
        print("result :", self.result)
        print("guesses:", len(self.history))
        print("----------------------")

    def run(self):
        print(f"Wordle - {self.length} letters, {self.max_tries}
guesses. q to quit.")
        print("[X] right spot, (X) wrong spot, plain = not in word")
        while len(self.history) < self.max_tries:
            left = self.max_tries - len(self.history)
            guess = input(f"[{left} left] > ").strip().lower()
            if guess == "q":
                self.result = "quit"
                break
            if len(guess) != self.length or not guess.isalpha():
                print(f"enter a {self.length} letter word, letters
only.")
                continue
            self.history.append((guess, evaluate(self.target, guess)))
            self.show_history()
            if guess == self.target:
                self.result = "won"
                break
        else:
            self.result = "lost"
```

```
        self.show_summary()
        return self.result
```

`main.py` (full file):

python

```
from game import WordleGame


def pick_length():
    while True:
        raw = input("word length (4/5/6): ").strip()
        if raw in ("4", "5", "6"):
            return int(raw)
        print("pick 4, 5 or 6")


def session_summary(games):
    wins = [g for g in games if g.result == "won"]
    losses = [g for g in games if g.result == "lost"]
    quits = [g for g in games if g.result == "quit"]
    print()
    print("======== session summary ========")
    print(f"games: {len(games)}  won: {len(wins)}  lost: {len(losses)}
quit: {len(quits)}")
    if wins:
        avg = sum(len(g.history) for g in wins) / len(wins)
        print(f"avg guesses on wins: {avg:.1f}")
    for i, g in enumerate(games, 1):
        print(f"{i}. {g.length} letters | {g.target} | {g.result} |
{len(g.history)} guesses")
    print("================================")


def main():
    games = []
    while True:
        game = WordleGame(pick_length())
        game.run()
        games.append(game)
        if input("play again? (y/n) ").strip().lower() != "y":
            break
    session_summary(games)


if __name__ == "__main__":
    main()
```

bash

```
git add game.py main.py
git commit -m "task 4: guess history and session summary"
git push origin main
```

"After" video + testing

A random target makes win/loss demos awkward, so pin the target without editing any code:

bash

```
python -c "from game import WordleGame; g=WordleGame(5);
g.target='crane'; g.run()"
```

Then type `eerie` (should show `E`, `E` plain, `(R)`, `I` plain, `[E]`), then `abc` (invalid, no turn used, no history reprint), then `crane` to win. Do a second one with 5 wrong guesses, then the right one, to show winning on the last guess. For the 10 seconds, show: pinned run with eerie, invalid input, win. Do the rest of the checklist (4 and 6 letter modes, loss, quit, play again + session summary) separately via `python main.py` and just run through it yourself for testing.

Quick sanity asserts if you want them, in a `test_feedback.py` you can commit too:

python

```
from feedback import evaluate

assert evaluate("crane", "eerie") == ["gray", "gray", "yellow",
"gray", "green"]
assert evaluate("apple", "paper") == ["yellow", "yellow", "green",
"yellow", "gray"]
assert evaluate("moon", "noon") == ["gray", "green", "green", "green"]
assert evaluate("stone", "stone") == ["green"] * 5
print("ok")
```

Submission

bash

```
mkdir Lab-4
cp main.py game.py feedback.py words.py requirements.txt README.md
Lab-4/
```

Drop `before.mp4`, `after.mp4` and the chat PDF into `Lab-4/` too, then:

bash

```
git add Lab-4
git commit -m "lab 4 deliverables"
git push origin main
```

For the chat PDF, export this chat (browser print, save as PDF) after you've gone through the prompts above, since they want a PDF and not a link. Run each stage yourself and read the explanations before committing, since the lab is graded on you understanding it.



Here’s my wordle project (main.py, game.py, feedback.py, words.py). Read through it and explain what each file does and where you think the bugs are. Don’t change anything yet.

No new files came through this turn, so I'm going off the screenshots from earlier (the original repo code). If you've edited anything since, paste it and I'll recheck.

main.py is just the entry point. It imports `WordleGame` and calls `WordleGame().run()`. It's hardcoded to the default 5 letters, there's no way to pick a length, and it quits after one game.

words.py is a list of 10 words, all 5 letters. That means 4- and 6-letter modes can't work yet, since `random.choice` on an empty filtered list would crash with an IndexError. It's a small list, but apple, array and eerie have repeated letters, which is handy for testing.

game.py holds the `WordleGame` class. `__init__` stores the length, picks a random target of that length, and sets up an empty history list. `run()` loops 6 times, reads a guess, and quits on `q`. For anything not the right length or not alphabetic, it prints an error and `continue`s. Otherwise it calls `evaluate`, appends `(guess, feedback)` to history, prints the feedback, and returns on a win. If the loop runs out, it reveals the word.

Bugs I see in here:

- Invalid guesses burn a turn. The `for _ in range(6)` counts every iteration, including the `continue` ones, so typing junk six times ends the game with no real guesses made.
- History gets filled but never displayed, and there's no final summary.
- Feedback prints as `green yellow gray gray green`, which is hard to read next to the guess. It doesn't even show the letters.
- No win/loss result is returned, and `q` quits silently without revealing the word.
- It never shows how many guesses are left, even though the header says 6.
- Ctrl+C or Ctrl+D gives an ugly traceback.

feedback.py has `evaluate(target, guess)`. It starts with all gray, marks exact matches green in the first pass, then in the second pass marks yellow anything that's `in target`. That's the main bug: `ch in target` ignores how many copies exist. Guess `eerie` against `crane` and the first two `e`s come out yellow even though the target's only `e` was already used by the green one. A correct version counts the target's unmatched letters and uses each one up at most once. The stray half-comment on line 3 (`# occurrence, producing...`) looks like a truncated hint left behind. It's harmless, but worth deleting. It also assumes both strings are the same length, which is fine only because `game.py` checks first.

requirements.txt is only comments (stdlib only, Python 3.9+). Nothing to fix there.

One minor repo thing: the `__pycache__` folder is committed, so add a `.gitignore` for it at some point.

Tell me which one you want fixed first and I'll make the change.

`evaluate` in feedback.py gives wrong yellows for repeated letters. Fix it with standard wordle rules: resolve greens first, and each letter in the target can only be used once. Only change feedback.py, keep the same function signature and return format.

Here's the whole of `feedback.py`:

python

```
def evaluate(target, guess):
    result = ["gray"] * len(guess)
    left = {}
    for i, ch in enumerate(guess):
        if ch == target[i]:
            result[i] = "green"
        else:
            left[target[i]] = left.get(target[i], 0) + 1
    for i, ch in enumerate(guess):
        if result[i] == "green":
            continue
        if left.get(ch, 0) > 0:
            result[i] = "yellow"
            left[ch] -= 1
    return result
```

First pass marks the greens and counts the target letters that didn't get matched exactly. Second pass hands out yellows left to right, and each yellow uses up one of those counted letters, so a letter can never be yellow more times than it's actually left in the target. The signature and return format are the same, and I dropped the leftover half-comment.

Check it:

bash

```
python -c "from feedback import evaluate;
print(evaluate('crane','eerie'))"
python -c "from feedback import evaluate;
print(evaluate('apple','paper'))"
python -c "from feedback import evaluate;
print(evaluate('moon','noon'))"
```

Expected:

```
['gray', 'gray', 'yellow', 'gray', 'green']
['yellow', 'yellow', 'green', 'yellow', 'gray']
['gray', 'green', 'green', 'green']
```

Then commit it:

bash

```
git add feedback.py
```

```
git commit -m "task 1: fix duplicate letter feedback"
```

In game.py, invalid guesses (wrong length or non-alpha) are using up one of the 6 turns because of the for loop. Change it so only accepted guesses count, enforce max 6 accepted guesses, and add clean win and loss messages. ‘q’ should still quit. Keep it in the WordleGame class.

Full `game.py`:

python

```
import random
from words import WORDS
from feedback import evaluate


class WordleGame:
    def __init__(self, length=5):
        self.length = length
        self.target = random.choice([w for w in WORDS if len(w) ==
length])
        self.history = []
        self.max_tries = 6
        self.result = None

    def run(self):
        print(f"Wordle - {self.length} letters, {self.max_tries}
guesses. q to quit.")
        while len(self.history) < self.max_tries:
            left = self.max_tries - len(self.history)
            guess = input(f"[{left} left] > ").strip().lower()
            if guess == "q":
                print("quit. the word was:", self.target)
                self.result = "quit"
                return self.result
            if len(guess) != self.length or not guess.isalpha():
                print(f"enter a {self.length} letter word, letters
only.")
                continue
            feedback = evaluate(self.target, guess)
            self.history.append((guess, feedback))
            print(" ".join(feedback))
            if guess == self.target:
                print(f"solved in {len(self.history)}!")
                self.result = "won"
                return self.result
        print("out of guesses. the word was:", self.target)
        self.result = "lost"
        return self.result
```

What changed: the `for _ in range(6)` is now a `while` that checks `len(self.history)`, and history only grows on accepted guesses, so invalid input hits `continue` without costing anything. The prompt shows guesses left, `q` reveals the word and returns `"quit"`, a win prints how many guesses it took, and running out prints the loss message with the word. `run()` returns `"won"`, `"lost"` or `"quit"` (also stored in `self.result`) so later tasks can use it.

Test with a pinned target so you can hit every case:

bash

```
python -c "from game import WordleGame; g=WordleGame(5);
g.target='crane'; g.run()"
```

- type `abc`, `12345`, empty enter: counter should stay at `[6 left]`
- type `eerie`: counter drops to `[5 left]`
- win: type `crane`
- loss: six valid wrong guesses, like `apple` six times
- win on the last guess: five wrong ones, then `crane`
- quit: `q` any time

Then commit:

bash

```
git add game.py
git commit -m "task 2: win/loss handling and guess limit"
```



Add 4, 5 and 6 letter modes. words.py only has 5-letter words, so add a handful of 4 and 6 letter words (include some with repeated letters). Ask for the length at startup in main.py and re-ask on invalid input. Don't touch feedback.py unless it breaks for other lengths.

`feedback.py` doesn't need touching. It works off `len(guess)` and never assumes 5, so it's fine for 4 and 6.

`words.py`:

python

```
WORDS = [
    "apple", "array", "eerie", "crane", "grape",
    "stare", "steam", "stone", "plant", "flame",
    "moon", "book", "door", "tree", "cake", "jazz",
    "coffee", "summer", "planet", "garden", "bridge", "silver",
]
```

Repeated letters in there: moon, book, door, tree, jazz (4), eerie, array, apple (5), coffee, summer (6).

`main.py`:

python

```
from game import WordleGame


def pick_length():
    while True:
        raw = input("word length (4/5/6): ").strip()
        if raw in ("4", "5", "6"):
            return int(raw)
        print("pick 4, 5 or 6")


if __name__ == "__main__":
    WordleGame(pick_length()).run()
```

`game.py`: two small edits. Add this under the imports:

python

```
LENGTHS = (4, 5, 6)
```

And make this the first thing inside `__init__`:

python

```
        if length not in LENGTHS:
            raise ValueError("length must be 4, 5 or 6")
```

That way a bad length fails loudly instead of crashing on an empty `random.choice`.

Test each mode with a pinned target, since the target is random otherwise:

bash

```
python main.py
python -c "from game import WordleGame; g=WordleGame(4);
g.target='moon'; g.run()"
python -c "from game import WordleGame; g=WordleGame(6);
g.target='coffee'; g.run()"
```

Things to try:

- startup: enter `3`, `7`, `abc` and an empty enter, and each should re-ask
- 4 letters vs `moon`: guess `noon`, expect gray, green, green, green
- 6 letters vs `coffee`: guess `effect`, should resolve the repeated `f`s and `e`s without extra yellows
- wrong-length guess in each mode (5 letters in a 4-letter game) should be rejected without costing a turn

Commit:

bash

```
git add words.py game.py main.py
git commit -m "task 3: 4/5/6 letter modes"
```



After every accepted guess, print the full history so far in a readable way (guess in caps plus a G/Y/- marker per letter). Invalid guesses must not add anything. When the game ends (win, loss or quit), print a summary with result, the word, and guesses used. Keep it modular, no files or extra libraries.

Only `game.py` changes. Full file:

python

```
import random
from words import WORDS
from feedback import evaluate

LENGTHS = (4, 5, 6)
MARKS = {"green": "G", "yellow": "Y", "gray": "-"}


def format_row(n, guess, feedback):
    marks = " ".join(MARKS[c] for c in feedback)
    return f"{n}. {guess.upper()}   {marks}"


class WordleGame:
    def __init__(self, length=5):
        if length not in LENGTHS:
            raise ValueError("length must be 4, 5 or 6")
        self.length = length
        self.target = random.choice([w for w in WORDS if len(w) ==
length])
        self.history = []
        self.max_tries = 6
        self.result = None

    def show_history(self):
        print()
        for n, (guess, feedback) in enumerate(self.history, 1):
            print(format_row(n, guess, feedback))
        print()

    def show_summary(self):
        print("---- game summary ----")
        print("result :", self.result)
        print("word   :", self.target.upper())
        print("guesses:", len(self.history))
        print("----------------------")

    def run(self):
        print(f"Wordle - {self.length} letters, {self.max_tries}
guesses. q to quit.")
        print("G = right spot, Y = wrong spot, - = not in word")
        while len(self.history) < self.max_tries:
            left = self.max_tries - len(self.history)
            guess = input(f"[{left} left] > ").strip().lower()
            if guess == "q":
```

```
                self.result = "quit"
                break
            if len(guess) != self.length or not guess.isalpha():
                print(f"enter a {self.length} letter word, letters
only.")
                continue
            self.history.append((guess, evaluate(self.target, guess)))
            self.show_history()
            if guess == self.target:
                self.result = "won"
                break
        else:
            self.result = "lost"
        self.show_summary()
        return self.result
```

How it behaves:

- `show_history()` only runs right after a guess gets appended, so invalid input prints nothing and adds nothing.
- Win, loss and quit all fall out of the loop and hit the same `show_summary()`, so the summary logic lives in one place. The `while ... else` sets `"lost"` only when the loop ends without a `break`.
- The old plain `green yellow gray` print is gone, since the history replaces it.
- `main.py` stays as it was.

Output looks like:

```
[6 left] > eerie

1. EERIE   - - Y - G

[5 left] > crane

1. EERIE   - - Y - G
2. CRANE   G G G G G

---- game summary ----
result : won
word   : CRANE
guesses: 2
----------------------
```

Test with a pinned target:

bash

```
python -c "from game import WordleGame; g=WordleGame(5);
g.target='crane'; g.run()"
```

- `eerie` should give `- - Y - G`
- `abc` and an empty enter should print only the error, with the counter unchanged and no history reprint
- win, six wrong guesses for a loss, and `q` (including `q` after a couple of guesses) should each end with the summary and the right result

Commit:

bash

```
git add game.py
git commit -m "task 4: guess history and game summary"
git push origin main
```



https://github.com/SETAPESU26/17_wordle
for games that do not have a UI, you can submit a screen recording of the terminal with the game running and clearly visible.
pls make sure all the tasks mentioned are clearly showcased in your videos (for eg: timer, score updates, etc)
pls ask everyone to submit a pdf of the chat history and not a link since i won't be able to view the link
So basically, i need to run this whole thing in terminal, cmd whatever, give me all the commands to run it starting from git clone, and also give the necessary changes to be done to the code, give the code changes prompt by prompt
Scenario 17 — Wordle
A terminal Wordle-style game with duplicate-letter scoring and guess history.
Provided files

* `main.py` — entry point.
* `game.py` — game loop and history.
* `feedback.py` — letter-evaluation rules.
* `words.py` — in-memory word list.
* `requirements.txt` — dependency declaration.

Setup

```bash
python main.py

```

Before changing the code
Read the feedback implementation carefully. Test guesses containing repeated letters
against targets that contain fewer copies of that letter than the guess.
Task 1 — Correct duplicate-letter feedback
Implement standard Wordle-style feedback in which exact matches are resolved first and
each occurrence in the target can satisfy at most one occurrence in the guess.
Done when: repeated-letter guesses produce correct exact/present/absent results.
Task 2 — Complete guess handling
Add clean win/loss handling, enforce the six accepted-guess limit, and ensure invalid
guesses do not consume a turn.
Task 3 — Variable word lengths
Add 4-, 5-, and 6-letter modes using the supplied in-memory words. The feedback logic
must work for all supported lengths.
Task 4 — Guess history and summary
Display a readable history after each accepted guess and provide a final session summary.
Do not duplicate entries for invalid attempts.
Required testing
Use repeated-letter cases, exact matches, absent letters, invalid lengths, all three

word lengths, winning on the final allowed guess, losing, and quitting.
LLM usage
You may use an LLM during the lab. The goal is to use it as a coding assistant while
retaining responsibility for understanding and testing the result.

* Inspect the existing code before asking for changes.
* Ask for explanations when you do not understand a proposed change.
* Test generated code against the stated behaviour and edge cases.
* Keep your complete LLM chat history for submission.
* Do not replace the whole project with an unrelated implementation.
* Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

* [ ] Task 1 completed and the original defect was reproduced and fixed.
* [ ] Tasks 2–4 completed and tested.
* [ ] Boundary and invalid-input cases tested.
* [ ] No unnecessary external dependencies added.
* [ ] No persistent storage added.
* [ ] Code remains understandable and modular.
* [ ] Complete LLM chat-history link included.

Folder structure

```text
scenario-05-wordle/
├── README.md
├── requirements.txt
├── main.py
├── game.py
├── feedback.py
└── words.py

```

Submission Checklist
Submission is only the following three things:

* [ ] A 10-second video of gameplay before your changes, showing the bug/broken behavior
* [ ] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
* [ ] The Chat/LLM used page link, with the complete chat history

Here's the whole flow, start to finish.

0. Setup + "before" video

Fork `SETAPESU26/17_wordle` to your own account on GitHub first (that's how you avoid raising a PR to the main repo), then:

```bash
git clone https://github.com/<your-username>/17_wordle.git
cd 17_wordle
python main.py
```

(use `py main.py` if `python` isn't recognised on Windows)

What to record in the before video (10 sec):

```bash
python -c "from feedback import evaluate; print(evaluate('crane','eerie'))"
```

Prints `['yellow', 'yellow', 'yellow', 'gray', 'green']`. That's the bug: crane has only one `e` and the last `e` already took it as green, so the first two should be gray and `r` is the only yellow.

Then `python main.py`, type `abc` a few times (invalid input eats a turn, second bug), then `q`.

Task 1: duplicate-letter feedback

Prompt:
> Read feedback.py. evaluate() marks a letter yellow if it's anywhere in the target, even when copies are already used up. Rewrite it with standard Wordle rules: greens first, then each remaining target letter can satisfy only one yellow. Keep the same signature and return a list of "green"/"yellow"/"gray". Explain the change briefly.

`feedback.py`:

```python
def evaluate(target, guess):
result = ["gray"] * len(guess)
```

left = {}
for i, ch in enumerate(guess):
if ch == target[i]:
result[i] = "green"
else:
left[target[i]] = left.get(target[i], 0) + 1
for i, ch in enumerate(guess):
if result[i] == "green":
continue
if left.get(ch, 0) > 0:
result[i] = "yellow"
left[ch] -= 1
return result
```

Re-run the one-liner. You should get `['gray', 'gray', 'yellow', 'gray', 'green']`.

```bash
git add feedback.py
git commit -m "task 1: fix duplicate letter feedback"
```

Task 2: win/loss, 6-guess limit, invalid guesses are free

Prompt:
> In game.py, run() uses `for _ in range(6)` so invalid guesses use up a turn. Switch to a loop that only counts accepted guesses (max 6), show guesses left, handle win, loss (reveal the word) and quitting with q, and have run() return "won"/"lost"/"quit". Don't touch the other files.

`game.py`:

```python
import random
from words import WORDS
from feedback import evaluate



class WordleGame:
def __init__(self, length=5):
self.length = length
self.target = random.choice([w for w in WORDS if len(w) == length])

```
self.history = []
self.max_tries = 6
self.result = None

def run(self):
print(f"Wordle - {self.length} letters, {self.max_tries} guesses. q to quit.")
while len(self.history) < self.max_tries:
left = self.max_tries - len(self.history)
guess = input(f"[{left} left] > ").strip().lower()
if guess == "q":
print("quit. the word was:", self.target)
self.result = "quit"
return self.result
if len(guess) != self.length or not guess.isalpha():
print(f"enter a {self.length} letter word, letters only.")
continue
feedback = evaluate(self.target, guess)
self.history.append((guess, feedback))
print(" ".join(feedback))
if guess == self.target:
print(f"solved in {len(self.history)}!")
self.result = "won"
return self.result
print("out of guesses. the word was:", self.target)
self.result = "lost"
return self.result
```

```bash
git add game.py
git commit -m "task 2: win/loss handling and guess limit"
```

Task 3: 4/5/6 letter modes

Prompt:
> Add 4 and 6 letter modes. Add some 4 and 6 letter words to words.py (include a few with repeated letters). main.py should ask for the length (4/5/6), re-ask on bad input, and pass it to WordleGame. Make WordleGame raise ValueError for unsupported lengths. Confirm evaluate() doesn't assume 5.

`words.py`:

```python
WORDS = [
"apple", "array", "eerie", "crane", "grape",
"stare", "steam", "stone", "plant", "flame",
"moon", "book", "door", "tree", "cake", "jazz",
"coffee", "summer", "planet", "garden", "bridge", "silver",
]
```

`game.py`, change only these bits: add `LENGTHS = (4, 5, 6)` under the imports, and at the top of `__init__`:

```python
if length not in LENGTHS:
raise ValueError("length must be 4, 5 or 6")
```

`main.py`:

```python
from game import WordleGame


def pick_length():
while True:
raw = input("word length (4/5/6): ").strip()
if raw in ("4", "5", "6"):
return int(raw)
print("pick 4, 5 or 6")


if __name__ == "__main__":
WordleGame(pick_length()).run()
```

```bash
git add words.py game.py main.py
git commit -m "task 3: 4/5/6 letter modes"
```

Task 4: history + session summary

Prompt:
> After every accepted guess, print the full history so far, readable without colours (green = [E], yellow = (E), gray = plain). Invalid guesses must not add history or reprint it. Add a per-game summary, and in main.py a play-again loop that prints a session summary at the end: games played, wins/losses/quits, average guesses on wins, and one line per game. Keep it modular, no files or extra libraries.

`game.py` (full file now):

```python
import random
from words import WORDS
from feedback import evaluate

LENGTHS = (4, 5, 6)


def render(guess, feedback):
parts = []
for ch, c in zip(guess.upper(), feedback):
if c == "green":
parts.append(f"[{ch}]")
elif c == "yellow":
parts.append(f"({ch})")
else:
parts.append(f" {ch} ")
return " ".join(parts)


class WordleGame:
def __init__(self, length=5):
if length not in LENGTHS:
raise ValueError("length must be 4, 5 or 6")
self.length = length
self.target = random.choice([w for w in WORDS if len(w) == length])
self.history = []
self.max_tries = 6
self.result = None

def show_history(self):
```

```
print()
for n, (guess, feedback) in enumerate(self.history, 1):
print(f"{n}. {render(guess, feedback)}")
print()

def show_summary(self):
print("---- game summary ----")
print("word :", self.target)
print("result :", self.result)
print("guesses:", len(self.history))
print("----------------------")

def run(self):
print(f"Wordle - {self.length} letters, {self.max_tries} guesses. q to quit.")
print("[X] right spot, (X) wrong spot, plain = not in word")
while len(self.history) < self.max_tries:
left = self.max_tries - len(self.history)
guess = input(f"[{left} left] > ").strip().lower()
if guess == "q":
self.result = "quit"
break
if len(guess) != self.length or not guess.isalpha():
print(f"enter a {self.length} letter word, letters only.")
continue
self.history.append((guess, evaluate(self.target, guess)))
self.show_history()
if guess == self.target:
self.result = "won"
break
else:
self.result = "lost"
self.show_summary()
return self.result
```
```

`main.py` (full file):

```python
from game import WordleGame


def pick_length():
```
```

```
while True:
raw = input("word length (4/5/6): ").strip()
if raw in ("4", "5", "6"):
return int(raw)
print("pick 4, 5 or 6")


def session_summary(games):
wins = [g for g in games if g.result == "won"]
losses = [g for g in games if g.result == "lost"]
quits = [g for g in games if g.result == "quit"]
print()
print("======== session summary ========")
print(f"games: {len(games)} won: {len(wins)} lost: {len(losses)} quit:
{len(quits)}")
if wins:
avg = sum(len(g.history) for g in wins) / len(wins)
print(f"avg guesses on wins: {avg:.1f}")
for i, g in enumerate(games, 1):
print(f"{i}. {g.length} letters | {g.target} | {g.result} | {len(g.history)} guesses")
print("================================")


def main():
games = []
while True:
game = WordleGame(pick_length())
game.run()
games.append(game)
if input("play again? (y/n) ").strip().lower() != "y":
break
session_summary(games)


if __name__ == "__main__":
main()
```

```bash
git add game.py main.py
git commit -m "task 4: guess history and session summary"
git push origin main
```

```

## "After" video + testing

A random target makes win/loss demos awkward, so pin the target without editing any code:

```bash
python -c "from game import WordleGame; g=WordleGame(5); g.target='crane'; g.run()"
```

Then type `eerie` (should show `E`, `E` plain, `(R)`, `I` plain, `[E]`), then `abc` (invalid, no turn used, no history reprint), then `crane` to win. Do a second one with 5 wrong guesses, then the right one, to show winning on the last guess. For the 10 seconds, show: pinned run with eerie, invalid input, win. Do the rest of the checklist (4 and 6 letter modes, loss, quit, play again + session summary) separately via `python main.py` and just run through it yourself for testing.

Quick sanity asserts if you want them, in a `test_feedback.py` you can commit too:

```python
from feedback import evaluate

assert evaluate("crane", "eerie") == ["gray", "gray", "yellow", "gray", "green"]
assert evaluate("apple", "paper") == ["yellow", "yellow", "green", "yellow", "gray"]
assert evaluate("moon", "noon") == ["gray", "green", "green", "green"]
assert evaluate("stone", "stone") == ["green"] * 5
print("ok")
```

Submission

```bash
mkdir Lab-4
cp main.py game.py feedback.py words.py requirements.txt README.md Lab-4/
```
```

Drop `before.mp4`, `after.mp4` and the chat PDF into `Lab-4/` too, then:

```bash
git add Lab-4
git commit -m "lab 4 deliverables"
git push origin main
```

For the chat PDF, export this chat (browser print, save as PDF) after you've gone through the prompts above, since they want a PDF and not a link. Run each stage yourself and read the explanations before committing, since the lab is graded on you understanding it.